

Jegyzőkönyv

Mobil programozás alapjai

Féléves feladat

Készítette: **Emődi Máté**

Neptunkód: **GT43W5**

Dátum: **2026. január**

Tartalomjegyzék

Tartalomjegyzék	2
Feladat leírása	3
Adatbázis réteg	3
Üzleti Logika (ViewModel)	5
Felhasználói Felület (Jetpack Compose)	6
Képernyőképek az alkalmazásról	7

Feladat leírása

A feladat egy natív Android alkalmazás készítése volt Kotlin nyelven. Az alkalmazás célja egy bevásárlólista kezelése, ahol a felhasználó termékeket vehet fel, módosíthat és törölhet. Az adatok tartós tárolását a Room ORM adatbázis biztosítja, a felhasználói felület pedig a modern Jetpack Compose technológiával készült.

Adatbázis réteg

Az adatok tárolásához a Room könyvtárat használtam, amely egy absztrakciós réteg az SQLite felett. Három fő komponenst hoztam létre:

A) Entitás (Entity)

A ShoppingItem osztály reprezentálja az adatbázis tábláját (shopping_items). Minden bevásárlólistás tételnek van egy egyedi azonosítója, neve és mennyisége.

```
@Entity(tableName = "shopping_items")
```

```
data class ShoppingItem(
```

```
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
```

```
    val name: String,
```

```
    val quantity: String
```

```
)
```

B) Adatbázis Elérés (DAO)

A ShoppingDao interfész tartalmazza az SQL műveleteket. A Flow típust használtam a lekérdezésnél, így az adatok változása esetén a felület automatikusan frissül (reaktív programozás).

```
@Dao interface ShoppingDao {  
    @Query("SELECT * FROM shopping_items ORDER BY id DESC")  
    fun getAllItems(): Flow<List<ShoppingItem>>  
  
    @Insert(onConflict = OnConflictStrategy.REPLACE)  
    suspend fun insertItem(item: ShoppingItem)  
  
    @Update  
    suspend fun updateItem(item: ShoppingItem)  
  
    @Delete  
    suspend fun deleteItem(item: ShoppingItem)  
}
```

C) Adatbázis példány (Database)

A ShoppingDatabase osztály felelős az adatbázis létrehozásáért. Singleton (egyke) mintát alkalmaztam, hogy erőforrás-takarékos legyen, és egyszerre csak egy adatbázis-kapcsolat éljen.

```
@Database(entities = [ShoppingItem::class], version = 1, exportSchema = false)
abstract class ShoppingDatabase : RoomDatabase() {

    abstract fun dao(): ShoppingDao
}
```

Üzleti Logika (ViewModel)

A ShoppingViewModel köti össze az adatbázist a felülettel. A viewModelScope-ot használom, hogy a háttérszálon történő adatbázis műveletek (Coroutines) automatikusan leálljanak, ha az alkalmazás bezárul.

```
class ShoppingViewModel(application: Application) : AndroidViewModel(application) {

    private val dao = ShoppingDatabase.getDatabase(application).dao()

    // Az adatok folyamatos figyelése StateFlow segítségével

    val items = dao.getAllItems()

        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    fun addItem(name: String, quantity: String) {

        viewModelScope.launch { dao.insertItem(ShoppingItem(name = name,
quantity = quantity)) }

    } // Törlés és módosítás függvények... }
```

Felhasználói Felület (Jetpack Compose)

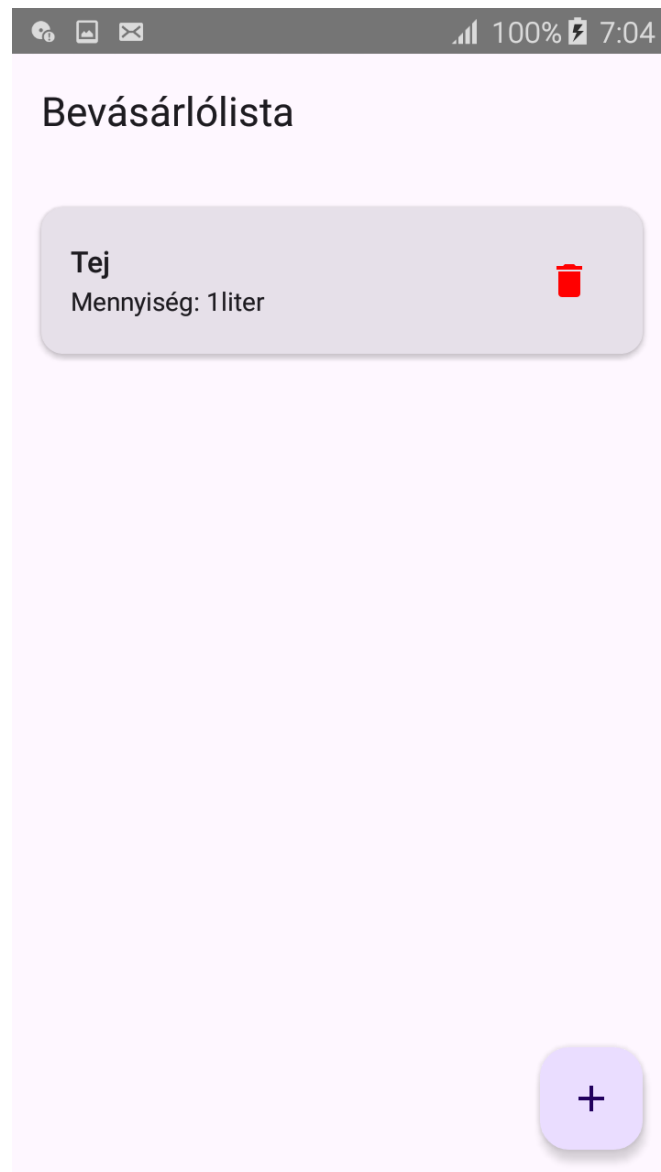
Az UI deklaratív módon készült. A ShoppingListApp a fő komponens, amely figyeli a ViewModel-ből érkező listát (collectAsState).

- Lista megjelenítése: LazyColumn (ez a modern ListView/RecyclerView megfelelője).
- Új elem felvétele: FloatingActionButton és egy felugró AlertDialog.

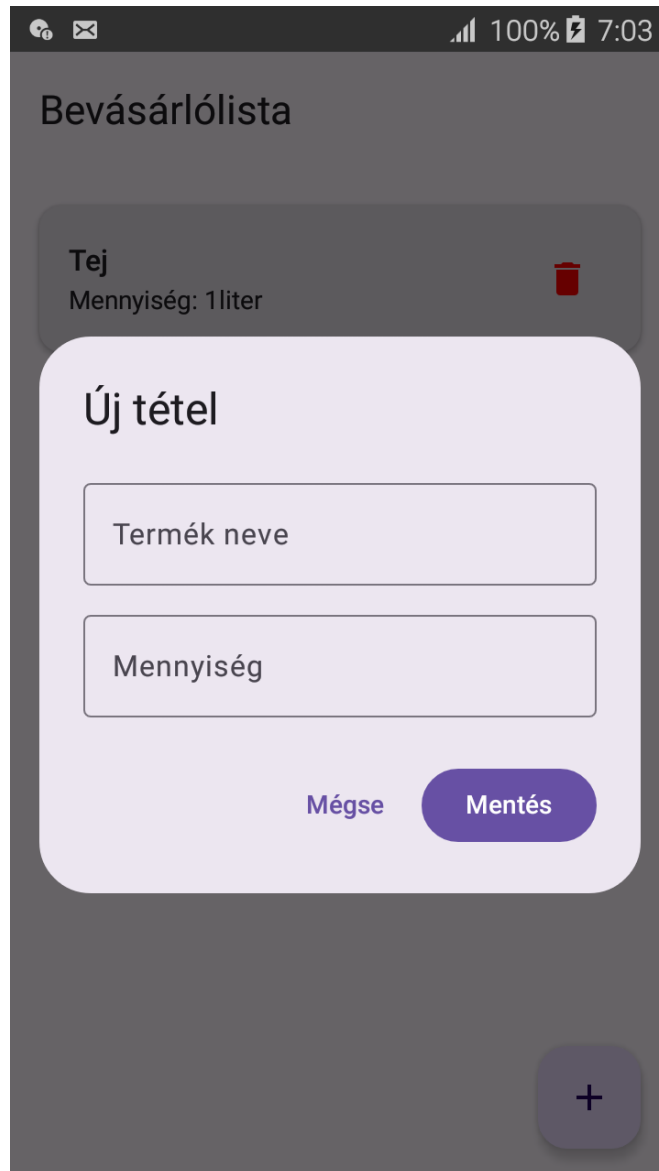
@Composable

```
fun ShoppingListApp(viewModel: ShoppingViewModel = viewModel()) {  
    val items by viewModel.items.collectAsState()  
    Scaffold(  
        floatingActionButton = {  
            FloatingActionButton(onClick = { /* Dialog megnyitása */ }) {  
                Icon(Icons.Default.Add, contentDescription = "Hozzáadás")  
            }  
        }  
    ) { padding ->  
        LazyColumn(modifier = Modifier.padding(padding)){  
            items(items) { item ->  
                ShoppingItemRow(item = item, ...)  
            }  
        }  
    }  
}
```

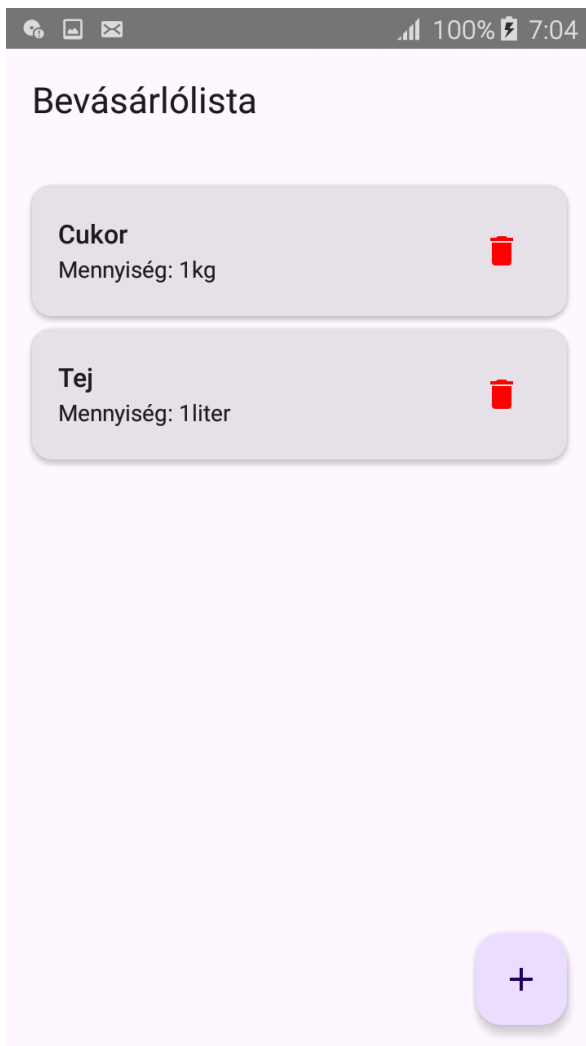
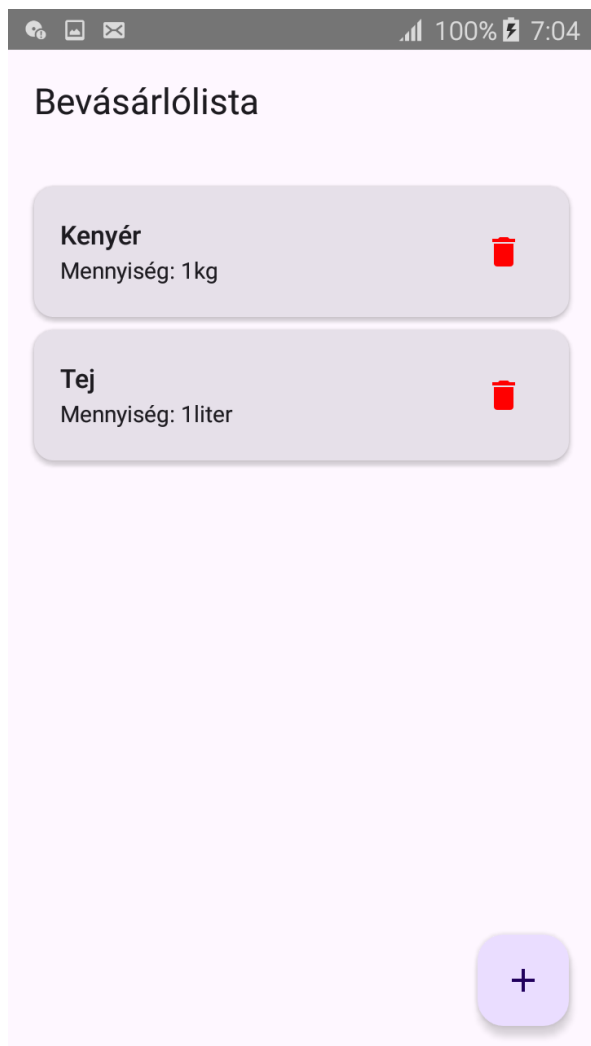
Képernyőképek az alkalmazásról



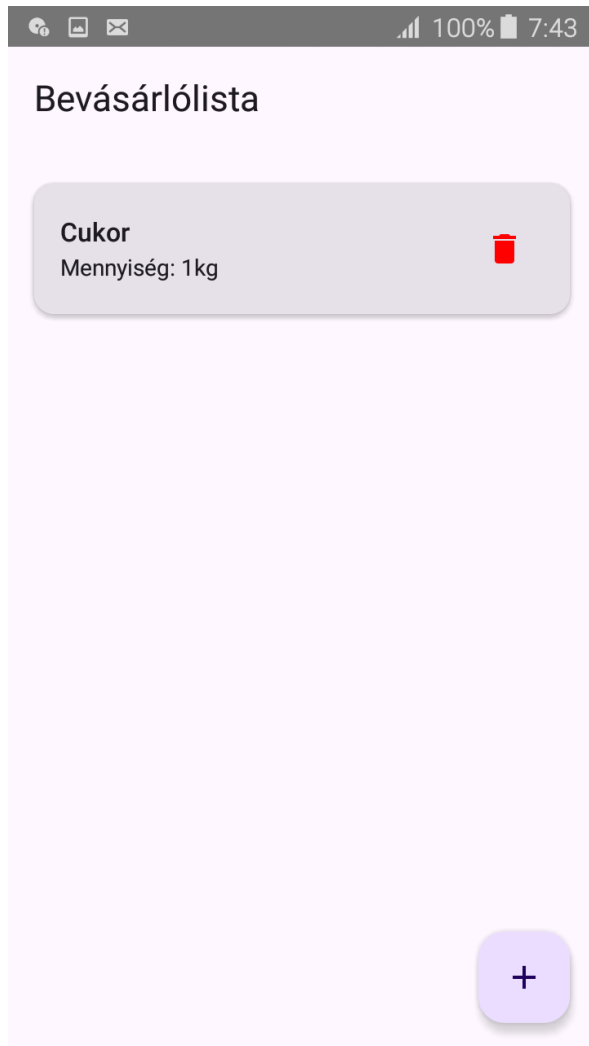
Tételek kilistázása



Új tétel hozzáadása



Tétel módosítása



Sikeres tétel törlés